CHAPTER 19

Google Maps

Map rendering, location tracking, route planning and ETA for a billion users.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a simplified Google Maps: **location update, navigation (route planning), ETA, and map rendering**. The challenge combines geospatial data modelling, routing algorithms on huge graphs, map tile serving, and real-time traffic.

Why interviewers ask it: Tests how to split a large problem (tiles / routing / location) and which data structures and algorithms (graph, Dijkstra, A*, hierarchical routing) apply.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Features?	Location update, navigation + ETA, map rendering.
Scale?	1 B DAU; 0.1 B active navigation users.
Road data?	Terabytes of road data from various sources.
Traffic conditions?	Consider real-time traffic.
Multiple travel modes and stops?	Driving, walking, transit; multi-stop is out of scope.

3. Requirements

Functional

- Show the map at any zoom level.
- Compute a route and ETA between A and B (with alternatives).
- Track a user's location during navigation.

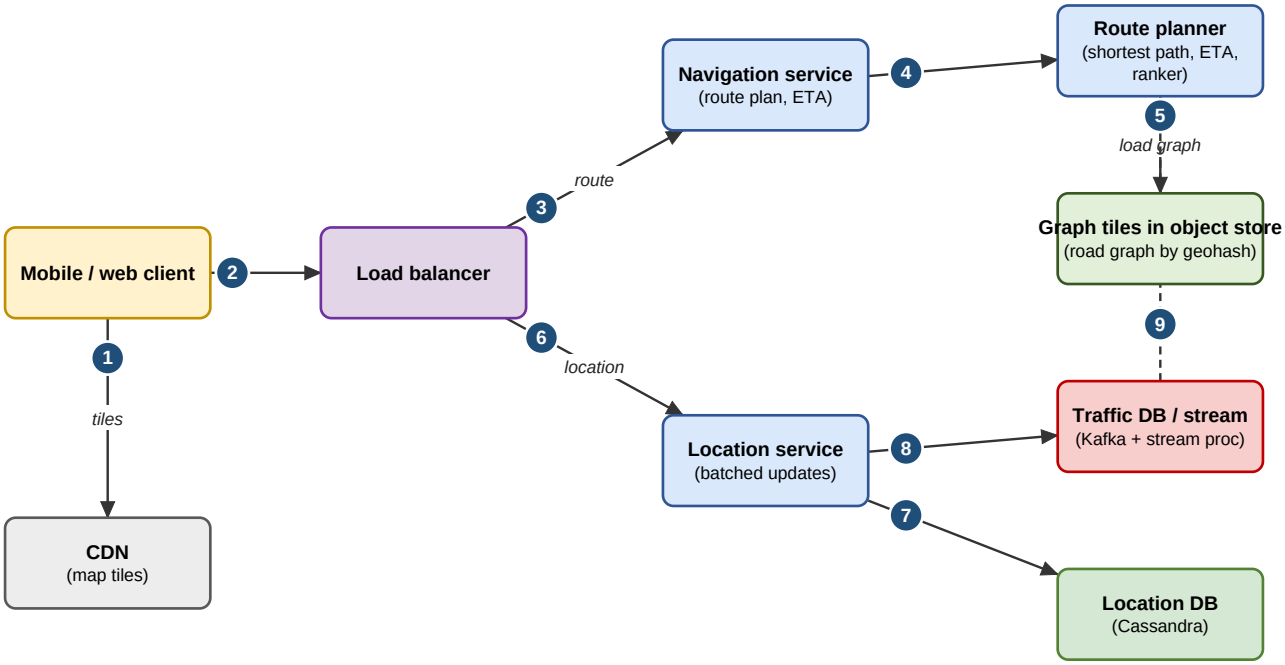
Non-functional

- Smooth map rendering (cached tiles).
- Accurate routes and ETA.
- Efficient use of mobile battery and data.
- High availability.

4. Back-of-the-envelope estimation

- Tiles: zoom level z has 4^z tiles, so zoom 21 alone has about 4.4 trillion. At ~100 KB each that is hundreds of PB raw, but most tiles (ocean, empty land) are identical or omitted; expect **tens of PB** across all zoom levels -> object storage + CDN, never the app servers.
- Navigation load: 1 B DAU x 35 min/week = about 5 B navigation-minutes/day -> about **3.5 M concurrent navigating users** on average.
- Location writes: 3.5 M users sending one batched update every 15 s = about **230 K QPS**; peak about 5x = **1.2 M QPS** -> Kafka + Cassandra (write-optimised), not a relational DB.
- Tile requests are absorbed by the CDN (target > 95% hit rate); origin handles only misses.

5. High-level design



Step	What happens
1	**Map rendering:** the client requests only the **tiles** for the viewport (zoom, x, y). They are static, so a CDN serves almost all of them; vector tiles are drawn on the device.
2	**Navigation:** the client sends origin and destination through the load balancer.
3	The navigation service validates the request and asks the route planner for candidate routes.
4	The **route planner** runs the shortest-path search (A* / hierarchical routing), ranks routes and computes ETA.
5	It loads only the needed **graph tiles** (road network by geohash, at several hierarchy levels) from object storage, lazily.
6	While navigating, the client sends **batched GPS samples** (about every 15 s) to the location service.
7	Samples are written to the **location DB** (Cassandra) for history and ML.
8	They are also streamed (Kafka) into **traffic processing**, which updates per-road-segment speeds.
9	Live traffic feeds ETA and reroutes; affected navigating users are notified over WebSocket / push.

6. Deep dive

Map rendering with tiles

- Pre-render the world into **tiles** at each zoom level (e.g. 256x256 px images); client requests only tiles for the viewport.
- Zoom level z has 4^z tiles; clients fetch via URL with (z, x, y) -> CDN-cacheable static content. **Vector tiles** (Mapbox) send geometry and render on device: smaller, scalable, smooth zoom.
- Tiles are produced offline from the road/geo data; updated rarely (daily/weekly).

Road data and graph model

- Roads = **graph** (intersections = nodes, road segments = edges with length, speed, restrictions). Raw data from satellite, crowd-sourcing, partners, GPS traces; processed by a **routing tile processing** pipeline into graph tiles stored by geohash in object storage (S3).
- Tiles are loaded on demand during path finding; only the needed area is in memory.

Routing algorithm

- Dijkstra / **A*** with a heuristic (straight-line distance) work for small graphs but are too slow for continent-sized graphs.
- **Hierarchical routing:** the graph is divided into levels (local roads, arterials, highways); route search at the start/end uses detailed tiles, the middle uses abstracted highway tiles. Also contraction hierarchies / bidirectional search.
- Routes are computed lazily: expand tiles near the origin and destination and connect through higher levels.

ETA and ranking

- ETA from distance + speed + **live traffic** + historical patterns (time-of-day). ML models refine.
- Return several routes; **ranker** applies user filters (avoid tolls/highways) and ranks by time.

Location service and traffic

- Clients batch GPS samples (e.g. every 15 s) to reduce load; location stream goes to Kafka -> real-time traffic aggregation (Flink) updates live traffic DB per road segment; also feeds historical DB for ML.
- Location DB: Cassandra (write-heavy, key = user id, timestamp).

Rerouting and adaptive ETA

- While navigating, the server tracks the user's route and active traffic changes; on a significant change, recompute and push a reroute. Keep a mapping from road segments to active navigating users to notify only those affected (a hashing/inverted index by tile).

Delivery protocol

- Mobile push via WebSocket (preferred, bi-directional) or long polling; fallback to periodic polling.

Offline maps and cost

- Pre-download tiles for areas; cache aggressively; use CDN to cut latency and egress.

7. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Tiles	Raster tiles / vector tiles	Vector: smaller, smoother zoom, device renders; raster simpler.
Path algorithm	Dijkstra / A* / hierarchical	Hierarchical with A* for large graphs.
Precompute vs on-demand	Precompute all-pairs routes / compute per request	On-demand hierarchical; cache popular routes.
Delivery	WebSocket / long polling	WebSocket for rerouting pushes.

8. Scaling, reliability, failure modes

- Routing is CPU bound -> stateless routing pods autoscale; graph tiles cached in memory and fetched from S3 on demand.
- Location writes: Kafka + Cassandra sharded by user.
- Multi-region for latency; tiles via CDN.

9. Likely follow-up questions

- How do you compute ETA with live traffic and ML?
- How do you handle road closures in near real time?
- How would you support transit or multi-stop routes?
- How do you decide tile sizes and zoom granularity?

10. Common mistakes

- Running plain Dijkstra over the entire world graph.
- Serving tiles from origin without a CDN.
- Ignoring client-side batching of GPS data.

11. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"Scope: map rendering, location updates, navigation with ETA and alternative routes, driving first. About 1 B DAU and 5 B navigation minutes a day means roughly 3.5 M concurrent navigators, so batched location writes are about 230 K QPS and over 1 M at peak. Tiles are tens of PB, so they must come from a CDN."
2. High-level design	10 min	"Three subsystems. Map rendering serves pre-generated tiles from a CDN. Navigation: a navigation service calls a route planner that loads graph tiles and returns ranked routes with ETA. Location: clients batch GPS points to a location service that writes to Cassandra and streams into traffic processing."
3. Deep dive	25 min	"Routing: plain Dijkstra over the whole world is too slow, so I partition the road graph into geohash tiles at multiple levels and use hierarchical routing with A* - detailed tiles near the endpoints and highway-level tiles in the middle, loaded lazily. ETA combines distance, speed, live traffic and history. Rerouting uses a map from road segments to active navigators so I notify only affected users. I'd use vector tiles to cut bandwidth."
4. Wrap up	4 min	"I'd discuss ML-based ETA, road closures, transit and multi-stop routing, and offline maps. Operationally: CDN hit rate, route latency and traffic pipeline lag."

12. 60-second recap / practice checklist

- Three subsystems: map tiles (CDN), routing (hierarchical graph tiles + A*), location/traffic (Kafka + Cassandra + stream processing).
- ETA = graph + live traffic + history; reroute pushes through WebSocket.
- Vector tiles and geohash-addressed graph tiles keep everything cacheable.

13. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Uber H3 hexagonal index
• DeepMind: ETA prediction in Google Maps
• Mapbox vector tile specification
• OSRM routing engine